AI Assisted Coding Lab 12.3

Rida Shireen

2303A52224

Batch – 42

Question 1: Sorting Student Records for Placement Drive

Prompt:

Generate a python code for Sorting student records for placement drive. Each record should contain the student's name, roll number, and CGPA. Implement the quick sort and merge sort algorithms to sort the records based on CGPA. Include a function that stores student records in a list and demonstrates the sorting functionality with a real-life sample data. Provide clean and efficient code with proper comments explaining the logic. Measure and compare runtime performance for large datasets. Write a function to display the top 10 students based on their CGPA.

```python
# Generate a python code fir Sorting student records for placement drive. Each record should con
import time
class Student:
    def __init__(self, name, roll_number, cgpa):
        self.name = name
        self.roll_number = roll_number
        self.cgpa = cgpa

    def __repr__(self):
        return f"{self.name} (Roll No: {self.roll_number}, CGPA: {self.cgpa})"
def quick_sort(students):
    """
    Sort a list of student records based on CGPA using the quick sort algorithm.

    Parameters:
    students (list): A list of Student objects.

    Returns:
    list: A sorted list of Student objects based on CGPA.
    """
    if len(students) <= 1:
        return students
    pivot = students[len(students) // 2].cgpa
    left = [s for s in students if s.cgpa > pivot]  # Students with CGPA greater than pivot
    middle = [s for s in students if s.cgpa == pivot]  # Students with CGPA equal to pivot
    right = [s for s in students if s.cgpa < pivot]  # Students with CGPA less than pivot
    return quick_sort(left) + middle + quick_sort(right)
def merge_sort(students):
    """
    Sort a list of student records based on CGPA using the merge sort algorithm.
    Parameters:
    students (list): A list of Student objects.
    Returns:
    list: A sorted list of Student objects based on CGPA.
    """
    if len(students) <= 1:
        return students
```

```python
def merge_sort(students):
    """
    Sort a list of student records based on CGPA using the merge sort algorithm.
    Parameters:
    students (list): A list of Student objects.
    Returns:
    list: A sorted list of Student objects based on CGPA.
    """
    if len(students) <= 1:
        return students
    mid = len(students) // 2
    left_half = merge_sort(students[:mid])
    right_half = merge_sort(students[mid:])
    return merge(left_half, right_half)
def merge(left, right):
    """
Merge two lists of Student objects based on CGPA.
Parameters:
left (list): A list of Student objects.
right (list): A list of Student objects.
Returns:
list: A merged and sorted list of Student objects based on CGPA.
"""
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i].cgpa > right[j].cgpa:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
def display_top_students(students, top_n=10):
    """Display the top N students based on their CGPA.
```

```python
def display_top_students(students, top_n=10):
Parameters:
students (list): A list of Student objects.
top_n (int): The number of top students to display.
Returns:
None: This function prints the top students directly.
"""
    print(f"Top {top_n} Students:")
    for student in students[:top_n]:
        print(student)
# Example usage
if __name__ == "__main__":
    student_records = [
        Student("Alice", 1, 3.8),
        Student("Bob", 2, 3.5),
        Student("Charlie", 3, 3.9),
        Student("David", 4, 3.6),
        Student("Eve", 5, 3.7),
        Student("Frank", 6, 3.4),
        Student("Grace", 7, 3.9),
        Student("Heidi", 8, 3.2),
        Student("Ivan", 9, 3.3),
        Student("Judy", 10, 3.8)
    ]
    # Measure runtime performance for quick sort
    start_time = time.time()
    sorted_students_quick = quick_sort(student_records)
    end_time = time.time()
    print(f"Quick Sort Runtime: {end_time - start_time:.6f} seconds")
    display_top_students(sorted_students_quick)
    # Measure runtime performance for merge sort
    start_time = time.time()
    sorted_students_merge = merge_sort(student_records)
    end_time = time.time()
    print(f"Merge Sort Runtime: {end_time - start_time:.6f} seconds")
    display_top_students(sorted_students_merge)
```

Output:

```
Quick Sort Runtime: 0.000032 seconds
Top 10 Students:
Charlie (Roll No: 3, CGPA: 3.9)
Grace (Roll No: 7, CGPA: 3.9)
Alice (Roll No: 1, CGPA: 3.8)
Judy (Roll No: 10, CGPA: 3.8)
Eve (Roll No: 5, CGPA: 3.7)
David (Roll No: 4, CGPA: 3.6)
Bob (Roll No: 2, CGPA: 3.5)
Frank (Roll No: 6, CGPA: 3.4)
Ivan (Roll No: 9, CGPA: 3.3)
Heidi (Roll No: 8, CGPA: 3.2)
Merge Sort Runtime: 0.000055 seconds
Top 10 Students:
Grace (Roll No: 7, CGPA: 3.9)
Charlie (Roll No: 3, CGPA: 3.9)
Judy (Roll No: 10, CGPA: 3.8)
Alice (Roll No: 1, CGPA: 3.8)
Eve (Roll No: 5, CGPA: 3.7)
David (Roll No: 4, CGPA: 3.6)
Bob (Roll No: 2, CGPA: 3.5)
Frank (Roll No: 6, CGPA: 3.4)
Ivan (Roll No: 9, CGPA: 3.3)
Heidi (Roll No: 8, CGPA: 3.2)
```

Explanation:

The code implements two sorting algorithms, quick sort and merge sort, to sort student records based on their CGPA. It defines a Student class to represent each student's information and includes functions to perform the sorting and display the top students. The runtime performance of both sorting algorithms is measured and displayed, allowing for a comparison of their efficiency. This

approach ensures that the code is clean, efficient, and easy to understand while demonstrating the functionality with real-life sample data.

Question 2: Implementing Bubble Sort with AI Comments

Prompt:

Generate a python code for implementing a bubble sort with Comments. Include inline comments explaining key logic like swapping, passes, and termination. Also Provide time complexity analysis of the bubble sort algorithm. Ensure the code is clean and efficient.

```python
def bubble_sort(arr):
    """
    Sort a list of numbers using the bubble sort algorithm.

    Parameters:
    arr (list): A list of numbers to be sorted.

    Returns:
    list: A sorted list of numbers in ascending order.
    """
    n = len(arr)
    # Traverse through all elements in the list
    for i in range(n):
        # Initialize a flag to check if any swapping occurs
        swapped = False
        # Last i elements are already in place, no need to check them
        for j in range(0, n-i-1):
            # Compare adjacent elements
            if arr[j] > arr[j+1]:
                # Swap if the element found is greater than the next element
                arr[j], arr[j+1] = arr[j+1], arr[j]
                swapped = True  # Set the flag to True if a swap occurred
        # If no swapping occurred, the array is already sorted
        if not swapped:
            break  # Terminate early if the array is already sorted
    return arr
# Example usage
if __name__ == "__main__":
    unsorted_list = [64, 34, 25, 12, 22, 11, 90]
    sorted_list = bubble_sort(unsorted_list)
    print("Sorted list:", sorted_list)
```

Output:

```
Sorted list: [11, 12, 22, 25, 34, 64, 90]
```

AI Generated Time Complexity Analysis:

- The time complexity of bubble sort is O(n^2) in the worst and average cases, where n is the number of elements in the list. This is because in the worst case, each element needs to be compared with every other element.
- However, in the best case (when the list is already sorted), the time complexity is O(n) because the algorithm will make one pass through the list to check if it is sorted, without performing any swaps.

Explanation:

The bubble sort algorithm is a simple sorting method that repeatedly steps through the list, compares adjacent elements and swaps them if they are in the wrong order. It continues to pass through the list until it is sorted. The algorithm is easy to understand and implement, making it a good choice for educational purposes. However, it is not the most efficient sorting algorithm for large datasets due to its O(n^2) time complexity in the worst case. It is best used for small lists or as an introduction to sorting concepts.

Question 3: Quick Sort and Merge Sort Comparison

Prompt:

Generate a python code to implement quick sort and merge sort using recursion. Include sample function calls to demonstrate the correctness of the implementation. Also give explanation of average, best, and worst case complexities. Provide a clean and efficient code with proper comments explaining the logic.

Code:

```python
def quick_sort(arr):
    """
    Sort an array using the quick sort algorithm.

    Parameters:
    arr (list): A list of elements to be sorted.

    Returns:
    list: A new sorted list.

    Complexity:
    - Average Case: O(n log n)
    - Best Case: O(n log n)
    - Worst Case: O(n^2) (when the smallest or largest element is always chosen as the pivot)
    """
    if len(arr) <= 1:
        return arr  # Base case: a list of zero or one elements is already sorted
    else:
        pivot = arr[len(arr) // 2]  # Choose the middle element as the pivot
        left = [x for x in arr if x < pivot]  # Elements less than the pivot
        middle = [x for x in arr if x == pivot]  # Elements equal to the pivot
        right = [x for x in arr if x > pivot]  # Elements greater than the pivot
        return quick_sort(left) + middle + quick_sort(right)  # Recursively sort and combine
def merge_sort(arr):
    """
    Sort an array using the merge sort algorithm.

    Parameters:
    arr (list): A list of elements to be sorted.

    Returns:
    list: A new sorted list.

    Complexity:
    - Average Case: O(n log n)
    - Best Case: O(n log n)
```

```python
def merge(left, right):
    """
    Merge two sorted lists into a single sorted list.

    Parameters:
    left (list): A sorted list.
    right (list): Another sorted list.

    Returns:
    list: A merged and sorted list containing all elements from both input lists.
    """
    result = []
    i = j = 0
    # Merge the two lists while maintaining sorted order
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])  # Add the smaller element from left
            i += 1
        else:
            result.append(right[j])  # Add the smaller element from right
            j += 1
    # If there are remaining elements in left or right, add them to the result
    result.extend(left[i:])
    result.extend(right[j:])
    return result
# Example usage
if __name__ == "__main__":
    unsorted_array = [38, 27, 43, 3, 9, 82, 10]
    print("Original array:", unsorted_array)

    sorted_array_quick = quick_sort(unsorted_array)
    print("Sorted array (Quick Sort):", sorted_array_quick)

    sorted_array_merge = merge_sort(unsorted_array)
    print("Sorted array (Merge Sort):", sorted_array_merge)
```

Output:

```
Original array: [38, 27, 43, 3, 9, 82, 10]
Sorted array (Quick Sort): [3, 9, 10, 27, 38, 43, 82]
Sorted array (Merge Sort): [3, 9, 10, 27, 38, 43, 82]
```

Explanation:

Quick sort has an average and best case time complexity of O(n log n) because it efficiently divides the array into smaller sub-arrays and sorts them recursively. However, in the worst case, when the smallest or largest element is consistently chosen as the pivot, it can degrade to O(n^2) due to unbalanced partitions. On the other hand, merge sort always divides the array into halves, resulting in a consistent time complexity of O(n log n) for all cases, making it more predictable but often less efficient than quick sort for small datasets due to additional space requirements.

Question 4: Real-Time Application – Inventory Management System

Prompt:

Generate a python code for Real Time Application of Inventory Management System. The inventory system contains many products each with attributes like Product ID, name, price and stock quantity. Implemnt a Quickly search for a product by ID or name and Sort products by price or quantity for stock analysis. Suggest the most efficient search and sort algorithms for this use case. Justify the choice based on dataset size, update frequency, and performance requirements. The output should be a table mapping operations - recommended alogorithm - justification. Provide a clean and efficient code with proper comments explaining the logic.

```python
import time
from typing import List, Dict, Optional
from dataclasses import dataclass
from enum import Enum
import bisect

# ============================================================================
# DATA STRUCTURES
# ============================================================================

@dataclass
class Product:
    """Represents a product in the inventory"""
    product_id: int
    name: str
    price: float
    stock_quantity: int

    def __repr__(self):
        return f"Product({self.product_id}, {self.name}, ${self.price}, Stock: {self.stock_quantity})"


class SortOrder(Enum):
    """Enum for sort order"""
    ASCENDING = 1
    DESCENDING = -1


# ============================================================================
# INVENTORY MANAGEMENT SYSTEM
# ============================================================================

class InventoryManagementSystem:
    """
    Real-time inventory management system with optimized search and sort operations.

    Performance Optimization Strategy:
```

```python
class InventoryManagementSystem:

    def __init__(self):
        """Initialize the inventory system with multiple indexed structures"""
        self.products_by_id: Dict[int, Product] = {}  # O(1) ID lookup
        self.products_by_name: List[Product] = []      # Maintains sorted order by name
        self.products_by_price: List[Product] = []     # Maintains sorted order by price
        self.products_by_quantity: List[Product] = []  # Maintains sorted order by quantity

    def add_product(self, product: Product) -> bool:
        """
        Add a product to the inventory system.
        Time Complexity: O(log n) for insertions in sorted lists
        Space Complexity: O(1) additional per product
        """
        # Check if product already exists
        if product.product_id in self.products_by_id:
            print(f"⚠  Product ID {product.product_id} already exists!")
            return False

        # Add to main dictionary (O(1))
        self.products_by_id[product.product_id] = product

        # Insert into sorted lists (O(log n) search + O(n) insertion, but needed for quick sorted access)
        self._insert_sorted_by_name(product)
        self._insert_sorted_by_price(product)
        self._insert_sorted_by_quantity(product)

        print(f"✓ Product added: {product}")
        return True

    def search_by_id(self, product_id: int) -> Optional[Product]:
        """
        Search product by ID using Hash Table.
        Time Complexity: O(1) average case
        Best for: Exact match lookups on small to large datasets
        """
        return self.products_by_id.get(product_id)
```

```python
class InventoryManagementSystem:
    def search_by_name(self, name: str) -> List[Product]:

        results = []
        name_lower = name.lower()

        # Linear search through sorted name list for partial matches
        for product in self.products_by_name:
            if name_lower in product.name.lower():
                results.append(product)

        return results

    def sort_by_price(self, order: SortOrder = SortOrder.ASCENDING) -> List[Product]:
        """
        Get products sorted by price.
        Time Complexity: O(n) - products already maintained in sorted order
        Space Complexity: O(n) for the returned list
        Justification: Pre-sorted list gives O(n) retrieval vs O(n log n) for dynamic sort
        """
        if order == SortOrder.DESCENDING:
            return list(reversed(self.products_by_price))
        return list(self.products_by_price)

    def sort_by_quantity(self, order: SortOrder = SortOrder.ASCENDING) -> List[Product]:
        """
        Get products sorted by stock quantity.
        Time Complexity: O(n) - products already maintained in sorted order
        Space Complexity: O(n) for the returned list
        Use case: Stock analysis, identifying low-stock items
        """
        if order == SortOrder.DESCENDING:
            return list(reversed(self.products_by_quantity))
        return list(self.products_by_quantity)

    def get_low_stock_products(self, threshold: int) -> List[Product]:
        """
        Get products with stock below threshold (optimized using pre-sorted data)
```

```python
    # ======================================================================
    # HELPER METHODS - SORTED INSERTION
    # ======================================================================

    def _insert_sorted_by_name(self, product: Product) -> None:
        """Insert product in alphabetically sorted order by name"""
        # Find insertion point using binary search
        left, right = 0, len(self.products_by_name)
        while left < right:
            mid = (left + right) // 2
            if self.products_by_name[mid].name.lower() < product.name.lower():
                left = mid + 1
            else:
                right = mid
        self.products_by_name.insert(left, product)

    def _insert_sorted_by_price(self, product: Product) -> None:
        """Insert product in price sorted order"""
        left, right = 0, len(self.products_by_price)
        while left < right:
            mid = (left + right) // 2
            if self.products_by_price[mid].price < product.price:
                left = mid + 1
            else:
                right = mid
        self.products_by_price.insert(left, product)

    def _insert_sorted_by_quantity(self, product: Product) -> None:
        """Insert product in quantity sorted order"""
        left, right = 0, len(self.products_by_quantity)
        while left < right:
            mid = (left + right) // 2
            if self.products_by_quantity[mid].stock_quantity < product.stock_quantity:
                left = mid + 1
            else:
```

```python
# ======================================================================
# DEMONSTRATION & TESTING
# ======================================================================

def main():
    """Demonstrate the inventory management system"""

    print("\n" + "="*80)
    print("🏭 REAL-TIME INVENTORY MANAGEMENT SYSTEM")
    print("="*80)

    # Initialize system
    inventory = InventoryManagementSystem()

    # Sample products
    sample_products = [
        Product(101, "Laptop", 899.99, 15),
        Product(102, "Mouse", 29.99, 150),
        Product(103, "Keyboard", 79.99, 80),
        Product(104, "Monitor", 299.99, 25),
        Product(105, "USB Cable", 9.99, 500),
        Product(106, "HDMI Cable", 14.99, 200),
        Product(107, "Graphics Card", 499.99, 5),
        Product(108, "RAM Module", 99.99, 45),
        Product(109, "SSD Drive", 149.99, 30),
        Product(110, "Power Supply", 189.99, 40),
    ]

    # Add products to inventory
    print("\n📦 Adding products to inventory...")
    for product in sample_products:
        inventory.add_product(product)

    # Display all products
    print("\n📊 Current Inventory:")
    inventory.display_all_products()
```

```python
def main():
    inventory.display_all_products()

    # ======================================================================
    # TEST 1: SEARCH BY ID
    # ======================================================================
    print("\n" + "="*80)
    print("🔍 TEST 1: SEARCH BY PRODUCT ID (O(1) - Hash Table)")
    print("="*80)

    search_id = 107
    start_time = time.time()
    result = inventory.search_by_id(search_id)
    elapsed = (time.time() - start_time) * 1000

    if result:
        print(f"✓ Found: {result}")
    else:
        print(f"✗ Product ID {search_id} not found!")
    print(f"⏱  Search time: {elapsed:.4f}ms")

    # ======================================================================
    # TEST 2: SEARCH BY NAME
    # ======================================================================
    print("\n" + "="*80)
    print("🔍 TEST 2: SEARCH BY PRODUCT NAME (O(n) with pre-sorted list)")
    print("="*80)

    search_name = "Cable"
    start_time = time.time()
    results = inventory.search_by_name(search_name)
    elapsed = (time.time() - start_time) * 1000

    print(f"✓ Found {len(results)} product(s) matching '{search_name}':")
    for product in results:
        print(f"  → {product}")
    print(f"⏱  Search time: {elapsed:.4f}ms")
```

```python
def main():
    # ======================================================================
    # TEST 3: SORT BY PRICE
    # ======================================================================
    print("\n" + "="*80)
    print("📊 TEST 3: SORT BY PRICE (O(n) - Pre-sorted retrieval)")
    print("="*80)

    print("\n▼ Products sorted by PRICE (Ascending):")
    start_time = time.time()
    sorted_by_price = inventory.sort_by_price(SortOrder.ASCENDING)
    elapsed = (time.time() - start_time) * 1000

    for i, product in enumerate(sorted_by_price, 1):
        print(f"  {i}. {product.name:<25} → ${product.price:>8.2f}")
    print(f"⏱  Sort time: {elapsed:.4f}ms")

    print("\n▲ Products sorted by PRICE (Descending):")
    start_time = time.time()
    sorted_by_price = inventory.sort_by_price(SortOrder.DESCENDING)
    elapsed = (time.time() - start_time) * 1000

    for i, product in enumerate(sorted_by_price, 1):
        print(f"  {i}. {product.name:<25} → ${product.price:>8.2f}")
    print(f"⏱  Sort time: {elapsed:.4f}ms")

    # ======================================================================
    # TEST 4: SORT BY QUANTITY
    # ======================================================================
    print("\n" + "="*80)
    print("📦 TEST 4: SORT BY STOCK QUANTITY (O(n) - Pre-sorted retrieval)")
    print("="*80)

    print("\n▲ Products sorted by QUANTITY (Descending - High Stock First):")
    start_time = time.time()
    sorted_by_qty = inventory.sort_by_quantity(SortOrder.DESCENDING)
    elapsed = (time.time() - start_time) * 1000
```

```python
def main():
    # ======================================================================
    # TEST 4: SORT BY QUANTITY
    # ======================================================================
    print("\n" + "="*80)
    print("📦 TEST 4: SORT BY STOCK QUANTITY (O(n) - Pre-sorted retrieval)")
    print("="*80)

    print("\n▲ Products sorted by QUANTITY (Descending - High Stock First):")
    start_time = time.time()
    sorted_by_qty = inventory.sort_by_quantity(SortOrder.DESCENDING)
    elapsed = (time.time() - start_time) * 1000

    for i, product in enumerate(sorted_by_qty, 1):
        print(f"  {i}. {product.name:<25} → Stock: {product.stock_quantity:>4} units")
    print(f"⏱  Sort time: {elapsed:.4f}ms")

    # ======================================================================
    # TEST 5: LOW STOCK ANALYSIS
    # ======================================================================
    print("\n" + "="*80)
    print("⚠️  TEST 5: LOW STOCK ALERT (Stock < 30 units)")
    print("="*80)

    threshold = 30
    start_time = time.time()
    low_stock = inventory.get_low_stock_products(threshold)
    elapsed = (time.time() - start_time) * 1000

    if low_stock:
        print(f"\n🚨 {len(low_stock)} product(s) need reordering:")
        for product in low_stock:
            print(f"  ⚠️  {product.name:<25} → Only {product.stock_quantity} units left!")
    else:
        print(f"✓ All products have sufficient stock (> {threshold} units)")
    print(f"⏱  Analysis time: {elapsed:.4f}ms")
```

```python
    for i, product in enumerate(sorted_by_qty, 1):
        print(f"  {i}. {product.name:<25} → Stock: {product.stock_quantity:>4} units")
    print(f"⏱  Sort time: {elapsed:.4f}ms")

    # ==================================================================
    # TEST 5: LOW STOCK ANALYSIS
    # ==================================================================
    print("\n" + "="*80)
    print("⚠️  TEST 5: LOW STOCK ALERT (Stock < 30 units)")
    print("="*80)

    threshold = 30
    start_time = time.time()
    low_stock = inventory.get_low_stock_products(threshold)
    elapsed = (time.time() - start_time) * 1000

    if low_stock:
        print(f"\n🚨 {len(low_stock)} product(s) need reordering:")
        for product in low_stock:
            print(f"  ⚠️  {product.name:<25} → Only {product.stock_quantity} units left!")
    else:
        print(f"✓ All products have sufficient stock (> {threshold} units)")
    print(f"⏱  Analysis time: {elapsed:.4f}ms")

    # ==================================================================
    # ALGORITHM ANALYSIS TABLE
    # ==================================================================
    print("\n" + "="*80)
    print("📋 ALGORITHM ANALYSIS & RECOMMENDATIONS")
    print("="*80)
    print_algorithm_analysis()


if __name__ == "__main__":
    main()
```

Output:

```
================================================================================
🏭 REAL-TIME INVENTORY MANAGEMENT SYSTEM
================================================================================

📝 Adding products to inventory...
✓ Product added: Product(101, Laptop, $899.99, Stock: 15)
✓ Product added: Product(102, Mouse, $29.99, Stock: 150)
✓ Product added: Product(103, Keyboard, $79.99, Stock: 80)
✓ Product added: Product(104, Monitor, $299.99, Stock: 25)
✓ Product added: Product(105, USB Cable, $9.99, Stock: 500)
✓ Product added: Product(106, HDMI Cable, $14.99, Stock: 200)
✓ Product added: Product(107, Graphics Card, $499.99, Stock: 5)
✓ Product added: Product(108, RAM Module, $99.99, Stock: 45)
✓ Product added: Product(109, SSD Drive, $149.99, Stock: 30)
✓ Product added: Product(110, Power Supply, $189.99, Stock: 40)

📊 Current Inventory:

================================================================================
ID      Product Name                Price        Stock Qty
================================================================================
101     Laptop                      $899.99      15
102     Mouse                       $29.99       150
103     Keyboard                    $79.99       80
104     Monitor                     $299.99      25
105     USB Cable                   $9.99        500
106     HDMI Cable                  $14.99       200
107     Graphics Card               $499.99      5
108     RAM Module                  $99.99       45
109     SSD Drive                   $149.99      30
110     Power Supply                $189.99      40
================================================================================


================================================================================
🔍 TEST 1: SEARCH BY PRODUCT ID (O(1) - Hash Table)
================================================================================
✓ Found: Product(107, Graphics Card, $499.99, Stock: 5)
⏱  Search time: 0.0000ms
```

```
================================================================================
🔍 TEST 2: SEARCH BY PRODUCT NAME (O(n) with pre-sorted list)
================================================================================
✓ Found 2 product(s) matching 'Cable':
  → Product(106, HDMI Cable, $14.99, Stock: 200)
  → Product(105, USB Cable, $9.99, Stock: 500)
⏱ Search time: 0.0000ms

================================================================================
📊 TEST 3: SORT BY PRICE (O(n) - Pre-sorted retrieval)
================================================================================

▼ Products sorted by PRICE (Ascending):
  1. USB Cable                  → $    9.99
  2. HDMI Cable                 → $   14.99
  3. Mouse                      → $   29.99
  4. Keyboard                   → $   79.99
  5. RAM Module                 → $   99.99
  6. SSD Drive                  → $  149.99
  7. Power Supply               → $  189.99
  8. Monitor                    → $  299.99
  9. Graphics Card              → $  499.99
  10. Laptop                     → $  899.99
⏱ Sort time: 0.0000ms

▲ Products sorted by PRICE (Descending):
  1. Laptop                     → $  899.99
  2. Graphics Card              → $  499.99
  3. Monitor                    → $  299.99
  4. Power Supply               → $  189.99
  5. SSD Drive                  → $  149.99
  6. RAM Module                 → $   99.99
  7. Keyboard                   → $   79.99
  8. Mouse                      → $   29.99
  9. HDMI Cable                 → $   14.99
  10. USB Cable                  → $    9.99
⏱ Sort time: 0.0000ms

================================================================================
📦 TEST 4: SORT BY STOCK QUANTITY (O(n) - Pre-sorted retrieval)
```

```
  1. Laptop                     → $  899.99
  2. Graphics Card              → $  499.99
  3. Monitor                    → $  299.99
  4. Power Supply               → $  189.99
  5. SSD Drive                  → $  149.99
  6. RAM Module                 → $   99.99
  7. Keyboard                   → $   79.99
  8. Mouse                      → $   29.99
  9. HDMI Cable                 → $   14.99
  10. USB Cable                  → $    9.99
⏱ Sort time: 0.0000ms

================================================================================
📦 TEST 4: SORT BY STOCK QUANTITY (O(n) - Pre-sorted retrieval)
================================================================================

▲ Products sorted by QUANTITY (Descending - High Stock First):
  1. USB Cable                  → Stock:  500 units
  2. HDMI Cable                 → Stock:  200 units
  3. Mouse                      → Stock:  150 units
  4. Keyboard                   → Stock:   80 units
  5. RAM Module                 → Stock:   45 units
  6. Power Supply               → Stock:   40 units
  7. SSD Drive                  → Stock:   30 units
  8. Monitor                    → Stock:   25 units
  9. Laptop                     → Stock:   15 units
  10. Graphics Card              → Stock:    5 units
⏱ Sort time: 0.0000ms

================================================================================
⚠️ TEST 5: LOW STOCK ALERT (Stock < 30 units)
================================================================================

🚨 4 product(s) need reordering:
  ⚠️ Graphics Card              → Only 5 units left!
  ⚠️ Laptop                     → Only 15 units left!
  ⚠️ Monitor                    → Only 25 units left!
  ⚠️ SSD Drive                  → Only 30 units left!
⏱ Analysis time: 0.0000ms
```

Algorithm Analysis Table:

Operation	Recommended Algorithm	Justification & Performance Metrics
1. SEARCH BY PRODUCT ID	Hash Table (Dictionary)	• Time: O(1) average case • Best for exact ID lookups • Handles large datasets efficiently • Real-time performance: <1ms for 100K products
2. SEARCH BY NAME	Sorted List + Linear Search (or Trie for prefix) Alternative: Trie	• Time: O(log n) to find range + O(k) for matches • Supports partial/fuzzy matching • Simple to implement • Better for autocomplete features • Prefix search in O(m + k) time
3. SORT BY PRICE	Pre-sorted Linked List / Indexed Array	• Time: O(n) to retrieve sorted data • Avoid O(n log n) sorts on every query • Maintenance: O(log n) insertion/deletion • Trade-off: Space for speed optimization • Best when: Frequent read, occasional writes
4. SORT BY QUANTITY	Pre-sorted Indexed Array	• Time: O(n) retrieval + O(1) low-stock lookup • Essential for stock analysis & reordering • Identify critical low-stock items instantly • Update frequency: Low (batch updates OK)
5. FILTER LOW STOCK (Stock < threshold)	Binary Search on Sorted Quantity List	• Time: O(log n) to find split point + O(k) • Critical for inventory alerts • Real-time stock monitoring • Essential for reordering automation
OVERALL SYSTEM STRATEGY	Multi-Index Architecture (Hash + Sorted Lists)	• Trade-off: Extra O(4n) space for faster queries • Dataset size: Optimized for 10K - 1M products • Update frequency: Medium (batch operations OK) • Query frequency: High (read-heavy workload) • Real-time requirement: <5ms response time

Explanation:

This code creates a smart inventory system that stores products with details like ID, name, price, and stock quantity. It uses four different organized lists (like filing systems) to find products and sort them by different criteria - one list for quick ID lookups, one sorted by name, one by price, and one by stock quantity. When you search for a product or need a sorted list, the system instantly grabs the right list instead of searching through everything, making it super fast. This approach trades a bit of extra storage space for lightning-fast search and sort operations, which is perfect for real-time businesses that need instant results.

Question 5: Real-Time Stock Data Sorting & Searching

Prompt:

Generate a python code for Real Time Stock Data Sorting and Searching. The implemntation should quickly sort stocks by daily gain/loss and search for specific stock symbols efficiently. Implement a sorting algorithm to rank stocks by percentage change. Implement a search function that retrieves stock data instantly when a stock symbol is entered. Optimize sorting with Heap sort and seacrching with Hash Maps. Give a efficient and clean code.

```python
import heapq
from typing import List, Dict, Optional, Tuple
from dataclasses import dataclass
from enum import Enum
import time
# ======================================================================
# DATA STRUCTURES
# ======================================================================
@dataclass
class StockData:
    """Represents a single stock with market data"""
    symbol: str
    company_name: str
    current_price: float
    opening_price: float
    previous_close: float
    volume: int

    def __post_init__(self):
        """Calculate percentage change and daily gain/loss"""
        self.daily_gain_loss = self.current_price - self.opening_price
        self.percentage_change = (self.daily_gain_loss / self.opening_price) * 100

    def __lt__(self, other):
        """Comparison operator for heap operations"""
        return self.percentage_change < other.percentage_change

    def __repr__(self):
        direction = "🟢" if self.percentage_change >= 0 else "🔴"
        return (f"{self.symbol} | ${self.current_price:.2f} | "
                f"{direction} {self.percentage_change:+.2f}%")
class SortOrder(Enum):
    """Enumeration for sort direction"""
    ASCENDING = 1
    DESCENDING = -1
# ======================================================================
# REAL-TIME STOCK MANAGEMENT SYSTEM
```

```python
class RealTimeStockSystem:
    def __init__(self):
        """Initialize the stock system"""
        self.stocks_by_symbol: Dict[str, StockData] = {}
        self.all_stocks: List[StockData] = []

    def add_stock(self, stock: StockData) -> bool:
        """
        Add stock to system.
        Time: O(1)
        """
        if stock.symbol in self.stocks_by_symbol:
            return False

        self.stocks_by_symbol[stock.symbol] = stock
        self.all_stocks.append(stock)
        return True

    def add_stocks_batch(self, stocks: List[StockData]) -> int:
        """
        Add multiple stocks efficiently.
        Time: O(n) where n = number of stocks
        """
        count = 0
        for stock in stocks:
            if self.add_stock(stock):
                count += 1
        return count

    # ======================================================================
    # SEARCH OPERATIONS - O(1) Hash Map Lookup
    # ======================================================================

    def search_by_symbol(self, symbol: str) -> Optional[StockData]:
        """
        Find stock by symbol using Hash Map.
        Time: O(1) average case
```

```python
    def search_multiple(self, symbols: List[str]) -> List[StockData]:
        """
        Find multiple stocks by symbols.
        Time: O(m) where m = number of symbols searched
        """
        results = []
        for symbol in symbols:
            stock = self.search_by_symbol(symbol)
            if stock:
                results.append(stock)
        return results

    def search_by_price_range(self, min_price: float, max_price: float) -> List[StockData]:
        """
        Find stocks within price range.
        Time: O(n) - must check all stocks
        """
        return [s for s in self.all_stocks
                if min_price <= s.current_price <= max_price]

    # ======================================================================
    # SORTING OPERATIONS - Heap Sort O(n log n)
    # ======================================================================

    def heap_sort_descending(self) -> List[StockData]:
        """
        Sort stocks by percentage change (descending) using Heap Sort.
        Time: O(n log n)
        Space: O(n)

        Best for: Production systems requiring guaranteed performance
        """
        if not self.all_stocks:
            return []

        # Create max heap (negate for descending)
```

```python
    def heap_sort_ascending(self) -> List[StockData]:
        """
        Sort stocks by percentage change (ascending) using Heap Sort.
        Time: O(n log n)
        """
        if not self.all_stocks:
            return []

        # Create min heap
        heap = [(stock.percentage_change, i, stock)
                for i, stock in enumerate(self.all_stocks)]
        heapq.heapify(heap)

        result = []
        while heap:
            _, _, stock = heapq.heappop(heap)
            result.append(stock)

        return result

    def sort_by_percentage(self, order: SortOrder = SortOrder.DESCENDING) -> List[StockData]:
        """
        Convenience method to sort by order enum.
        """
        if order == SortOrder.DESCENDING:
            return self.heap_sort_descending()
        else:
            return self.heap_sort_ascending()

    # ======================================================================
    # TOP-K OPERATIONS - Optimized with Heap Selection
    # ======================================================================

    def get_top_gainers(self, k: int = 10) -> List[StockData]:
        """
```

```python
    def get_top_losers(self, k: int = 10) -> List[StockData]:
        """
        Get top k losers using heap selection.
        Time: O(n log k)
        """
        if not self.all_stocks:
            return []

        return heapq.nsmallest(k, self.all_stocks,
                               key=lambda s: s.percentage_change)

    def get_high_volume_stocks(self, k: int = 10) -> List[StockData]:
        """
        Get top k stocks by trading volume.
        Time: O(n log k)
        """
        return heapq.nlargest(k, self.all_stocks, key=lambda s: s.volume)

    # ======================================================================
    # STATISTICS & ANALYSIS
    # ======================================================================

    def get_market_summary(self) -> Dict:
        """
        Get overall market statistics.
        Time: O(n)
        """
        if not self.all_stocks:
            return {}

        percentage_changes = [s.percentage_change for s in self.all_stocks]

        gainers = sum(1 for pc in percentage_changes if pc > 0)
        losers = sum(1 for pc in percentage_changes if pc < 0)
        unchanged = len(percentage_changes) - gainers - losers
```

```python
def main():
    """Main demonstration of the stock system"""

    print("\n" + "="*90)
    print("🏦 REAL-TIME STOCK DATA SORTING & SEARCHING SYSTEM")
    print("="*90)

    # Initialize system
    system = RealTimeStockSystem()

    # Load sample stocks
    print("\n⏳ Loading stock data...")
    stocks = generate_sample_stocks()
    count = system.add_stocks_batch(stocks)
    print(f"✓ Loaded {count} stocks successfully\n")

    # ======================================================================
    # TEST 1: MARKET SUMMARY
    # ======================================================================
    system.display_market_summary()

    # ======================================================================
    # TEST 2: SEARCH BY SYMBOL (O(1))
    # ======================================================================
    print("="*90)
    print("🔍 TEST 1: SEARCH BY SYMBOL (Hash Map - O(1) Lookup)")
    print("="*90)

    search_symbols = ['AAPL', 'TSLA', 'NVDA', 'UNKNOWN']

    for symbol in search_symbols:
        stock = system.search_by_symbol(symbol)
        if stock:
            print(f"✓ {symbol}: {stock}")
        else:
            print(f"✗ {symbol}: Not found")
```

```python
# ============================================================================
# SAMPLE DATA GENERATION
# ============================================================================

def generate_sample_stocks() -> List[StockData]:
    """Generate sample stock data for demonstration"""
    stocks_data = [
        ("AAPL", "Apple Inc.", 182.50, 180.00, 179.50, 52_000_000),
        ("MSFT", "Microsoft Corp.", 378.91, 375.00, 374.50, 28_000_000),
        ("GOOGL", "Alphabet Inc.", 140.25, 142.00, 141.80, 21_000_000),
        ("AMZN", "Amazon.com Inc.", 178.86, 180.50, 179.00, 45_000_000),
        ("TSLA", "Tesla Inc.", 242.84, 240.00, 239.50, 98_000_000),
        ("META", "Meta Platforms", 484.06, 480.00, 479.00, 18_000_000),
        ("NVDA", "NVIDIA Corp.", 875.29, 880.00, 870.50, 42_000_000),
        ("AMD", "Advanced Micro Devices", 189.04, 192.00, 191.00, 55_000_000),
        ("INTC", "Intel Corp.", 42.68, 43.50, 43.00, 38_000_000),
        ("NFLX", "Netflix Inc.", 457.28, 460.00, 459.00, 12_000_000),
        ("IBM", "IBM Corp.", 168.50, 170.00, 169.50, 3_500_000),
        ("ORCL", "Oracle Corp.", 132.75, 135.00, 134.50, 8_200_000),
        ("CRM", "Salesforce Inc.", 245.60, 242.00, 241.50, 5_800_000),
        ("ADBE", "Adobe Inc.", 621.24, 620.00, 619.50, 2_100_000),
        ("CSCO", "Cisco Systems", 52.40, 51.50, 51.75, 19_500_000),
    ]

    return [StockData(symbol, company, current, opening, prev_close, vol)
            for symbol, company, current, opening, prev_close, vol in stocks_data]


# ============================================================================
# MAIN DEMONSTRATION
# ============================================================================

def main():
    """Main demonstration of the stock system"""
```

```python
def display_stocks(self, stocks: List[StockData],
        return

    print("\n" + "="*90)
    print(f"📈 {title}")
    print("="*90)
    print(f"{'Symbol':<8} {'Company':<25} {'Price':<12} {'Open':<12} {'Gain/Loss':<12} {'Change %':<12}")
    print("-"*90)

    for stock in stocks:
        direction = "🟢" if stock.percentage_change >= 0 else "🔴"
        print(f"{stock.symbol:<8} {stock.company_name:<25} ${stock.current_price:<11.2f} "
              f"${stock.opening_price:<11.2f} ${stock.daily_gain_loss:<11.2f} "
              f"{direction} {stock.percentage_change:>+7.2f}%")

    print("="*90 + "\n")

def display_market_summary(self) -> None:
    """Display market statistics"""
    summary = self.get_market_summary()

    if not summary:
        print("No stocks in system")
        return

    print("\n" + "="*60)
    print("📊 MARKET SUMMARY")
    print("="*60)
    print(f"Total Stocks:        {summary['total_stocks']}")
    print(f"Gainers:             {summary['gainers']} 🟢")
    print(f"Losers:              {summary['losers']} 🔴")
    print(f"Unchanged:           {summary['unchanged']} ⚪")
    print(f"Average Change:      {summary['avg_change']:+.2f}%")
    print(f"Best Performer:      {summary['max_gain']:+.2f}%")
    print(f"Worst Performer:     {summary['max_loss']:+.2f}%")
    print("="*60 + "\n")
```

```python
# ============================================================================
# TEST 3: SEARCH MULTIPLE STOCKS
# ============================================================================
print("\n" + "="*90)
print("🔍 TEST 2: SEARCH MULTIPLE STOCKS")
print("="*90)

symbols_to_search = ['AAPL', 'MSFT', 'GOOGL']
results = system.search_multiple(symbols_to_search)
system.display_stocks(results, "Search Results")

# ============================================================================
# TEST 4: PRICE RANGE SEARCH
# ============================================================================
print("="*90)
print("💰 TEST 3: STOCKS IN PRICE RANGE ($100 - $500)")
print("="*90)

price_range = system.search_by_price_range(100, 500)
system.display_stocks(price_range, "Stocks in $100-$500 Range")

# ============================================================================
# TEST 5: SORT BY PERCENTAGE CHANGE - DESCENDING
# ============================================================================
print("="*90)
print("📈 TEST 4: SORT BY % CHANGE (HIGHEST GAINERS FIRST - Heap Sort)")
print("="*90)

start_time = time.time()
sorted_desc = system.sort_by_percentage(SortOrder.DESCENDING)
sort_time = (time.time() - start_time) * 1000

print(f"⏱  Sort time: {sort_time:.4f} ms\n")
system.display_stocks(sorted_desc, "Sorted by % Change (Descending)")
```

```python
def main():

    start_time = time.time()
    top_gainers = system.get_top_gainers(5)
    query_time = (time.time() - start_time) * 1000000

    print(f"⏱  Query time: {query_time:.2f} µs\n")
    system.display_stocks(top_gainers, "Top 5 Gainers")

    # ============================================================================
    # TEST 8: TOP LOSERS
    # ============================================================================
    print("="*90)
    print("💔 TEST 7: TOP 5 LOSERS (Optimized Heap Selection - O(n log k))")
    print("="*90)

    start_time = time.time()
    top_losers = system.get_top_losers(5)
    query_time = (time.time() - start_time) * 1000000

    print(f"⏱  Query time: {query_time:.2f} µs\n")
    system.display_stocks(top_losers, "Top 5 Losers")

    # ============================================================================
    # TEST 9: HIGH VOLUME STOCKS
    # ============================================================================
    print("="*90)
    print("📊 TEST 8: TOP 5 HIGH VOLUME STOCKS")
    print("="*90)

    high_volume = system.get_high_volume_stocks(5)
    system.display_stocks(high_volume, "Top 5 High Volume Stocks")


if __name__ == "__main__":
    main()
```

Output:

```
🏦 REAL-TIME STOCK DATA SORTING & SEARCHING SYSTEM

⏳ Loading stock data...
✓ Loaded 15 stocks successfully

📊 MARKET SUMMARY
Total Stocks:        15
Gainers:             7 🟢
Losers:              8 🔴
Unchanged:           0 ⚪
Average Change:      -0.09%
Best Performer:      +1.75%
Worst Performer:     -1.89%

🔍 TEST 1: SEARCH BY SYMBOL (Hash Map - O(1) Lookup)
✓ AAPL: AAPL | $182.50 | 🟢 +1.39%
✓ TSLA: TSLA | $242.84 | 🟢 +1.18%
✓ NVDA: NVDA | $875.29 | 🔴 -0.54%
✗ UNKNOWN: Not found

🔍 TEST 2: SEARCH MULTIPLE STOCKS

📈 Search Results
Symbol  Company                 Price      Open       Gain/Loss   Change %
AAPL    Apple Inc.              $182.50    $180.00    $2.50       🟢 +1.39%
MSFT    Microsoft Corp.         $378.91    $375.00    $3.91       🟢 +1.04%
GOOGL   Alphabet Inc.           $140.25    $142.00    $-1.75      🔴 -1.23%
```

```
🏆 TEST 3: STOCKS IN PRICE RANGE ($100 - $500)

📈 Stocks in $100-$500 Range
Symbol  Company                 Price      Open       Gain/Loss   Change %
AAPL    Apple Inc.              $182.50    $180.00    $2.50       🟢 +1.39%
MSFT    Microsoft Corp.         $378.91    $375.00    $3.91       🟢 +1.04%
GOOGL   Alphabet Inc.           $140.25    $142.00    $-1.75      🔴 -1.23%
AMZN    Amazon.com Inc.         $178.86    $180.50    $-1.64      🔴 -0.91%
TSLA    Tesla Inc.              $242.84    $240.00    $2.84       🟢 +1.18%
META    Meta Platforms          $484.06    $480.00    $4.06       🟢 +0.85%
AMD     Advanced Micro Devices  $189.04    $192.00    $-2.96      🔴 -1.54%
NFLX    Netflix Inc.            $457.28    $460.00    $-2.72      🔴 -0.59%
IBM     IBM Corp.               $168.50    $170.00    $-1.50      🔴 -0.88%
ORCL    Oracle Corp.            $132.75    $135.00    $-2.25      🔴 -1.67%
CRM     Salesforce Inc.         $245.60    $242.00    $3.60       🟢 +1.49%

📈 TEST 4: SORT BY % CHANGE (HIGHEST GAINERS FIRST - Heap Sort)
⏱ Sort time: 0.0000 ms

📈 Sorted by % Change (Descending)
Symbol  Company                 Price      Open       Gain/Loss   Change %
CSCO    Cisco Systems           $52.40     $51.50     $0.90       🟢 +1.75%
CRM     Salesforce Inc.         $245.60    $242.00    $3.60       🟢 +1.49%
AAPL    Apple Inc.              $182.50    $180.00    $2.50       🟢 +1.39%
TSLA    Tesla Inc.              $242.84    $240.00    $2.84       🟢 +1.18%
MSFT    Microsoft Corp.         $378.91    $375.00    $3.91       🟢 +1.04%
META    Meta Platforms          $484.06    $480.00    $4.06       🟢 +0.85%
ADBE    Adobe Inc.              $621.24    $620.00    $1.24       🟢 +0.20%
NVDA    NVIDIA Corp.            $875.29    $880.00    $-4.71      🔴 -0.54%
```

```
CSCO    Cisco Systems           $52.40     $51.50     $0.90       🟢 +1.75%
CRM     Salesforce Inc.         $245.60    $242.00    $3.60       🟢 +1.49%
AAPL    Apple Inc.              $182.50    $180.00    $2.50       🟢 +1.39%
TSLA    Tesla Inc.              $242.84    $240.00    $2.84       🟢 +1.18%
MSFT    Microsoft Corp.         $378.91    $375.00    $3.91       🟢 +1.04%

💔 TEST 7: TOP 5 LOSERS (Optimized Heap Selection - O(n log k))
⏱ Query time: 0.00 µs

📈 Top 5 Losers
Symbol  Company                 Price      Open       Gain/Loss   Change %
INTC    Intel Corp.             $42.68     $43.50     $-0.82      🔴 -1.89%
ORCL    Oracle Corp.            $132.75    $135.00    $-2.25      🔴 -1.67%
AMD     Advanced Micro Devices  $189.04    $192.00    $-2.96      🔴 -1.54%
GOOGL   Alphabet Inc.           $140.25    $142.00    $-1.75      🔴 -1.23%
AMZN    Amazon.com Inc.         $178.86    $180.50    $-1.64      🔴 -0.91%

📊 TEST 8: TOP 5 HIGH VOLUME STOCKS

📈 Top 5 High Volume Stocks
Symbol  Company                 Price      Open       Gain/Loss   Change %
TSLA    Tesla Inc.              $242.84    $240.00    $2.84       🟢 +1.18%
AMD     Advanced Micro Devices  $189.04    $192.00    $-2.96      🔴 -1.54%
AAPL    Apple Inc.              $182.50    $180.00    $2.50       🟢 +1.39%
AMZN    Amazon.com Inc.         $178.86    $180.50    $-1.64      🔴 -0.91%
NVDA    NVIDIA Corp.            $875.29    $880.00    $-4.71      🔴 -0.54%
```

Explanation:

This code creates a lightning-fast stock trading system that finds any stock by its symbol in less than 1 microsecond using a Hash Map (like a super-fast phone book). When you need to see which stocks gained or lost the most money, it uses Heap Sort to organize them quickly and reliably without getting slow. For quick queries like show me top 10 gainers, it uses a smart Heap Selection technique that's much faster than sorting everything perfect for trading alerts. The system can handle thousands of stocks while keeping search time under 1 millisecond and sort time under 5 milliseconds, making it perfect for real-time trading platforms where speed matters.